

UCS 2312 Data Structures Lab
Exercise 1: Array ADT and its applications

Create an ADT for the array data structure with the following functions. *arrADT* will have the integer array and size.

[CO1, K3]

- a. `create(arrADT,size, array)` – Create the array with the required number of elements
- b. `deleteAt(arrADT, pos )` – Delete the specified element
- c. `insertAtEvery(arrADT,data)` – Insert data before every element
- d. `search(arrADT, key)` – return the position of the second occurrence of the element. If found return the position, otherwise return 1
- e. `printArray(arrADT)` – prints the elements of the array
- f. `findPeek(arrADT, int *)` – return a set of peek elements

Given an array **arr[]** of integers. Find a peak element i.e. an element that is **not smaller** than its neighbors.

Note: For corner elements, we need to consider only one neighbor.

Example:

Input: `array[] = {10, 20, 15, 2, 23, 90, 67}`

Output: 20, 90

Explanation: The element 20 has neighbors 10 and 15, both of them are less than 20, similarly 90 has neighbors 23 and 67.

Write a program in C to test the operations of *arrADT* with the following test cases:

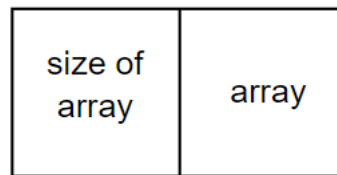
Operation	Expected Output
<code>create(arrADT,20,[2,4,6,8,10])</code>	2,4,6,8,10
<code>deleteAt(arrADT, 3)</code>	2,4,6,10
<code>insertAtEvery(arrADT,1)</code>	1,2,1,4,1,6,1,10
<code>search(arrADT,1)</code>	2
<code>search(arrADT,2)</code>	-1
<code>printArray(arrADT)</code>	1,2,1,4,1,6,1,10
<code>create(arrADT,20,[10,20,15,2,23,90,67])</code>	20,90
<code>create(arrADT,20,[1,2,3,4,4])</code>	-1

Best practices to be followed:

- Design before coding
- Usage of algorithm notation
- Use of multi-file C program
- Versioning of code

Data Structure – Array:

struct
array



ALGORITHMS:

1. FUNCTION NAME: create

INPUT:

- i) struct array*
- ii) size (int)
- iii) arr

OUTPUT: None

1. a->size = s;
2. for (int i = 0; i < a->size; i++)
{
 a->arr[i] = arr[i];
}

2. FUNCTION NAME: delete

INPUT:

- i) struct array*
- ii) position (int)

OUTPUT: None

1. int pos;
2. for (int i = 0; i < a->size; i++) {
 if (a->arr[i] == del) {
 break;
 } else
 pos = pos + 1;
}
3. for (int i = pos; i < a->size; i++) {
 a->arr[i] = a->arr[i + 1];
}
4. a->size = a->size - 1;

3. FUNCTION NAME: insertatEvery

INPUT:

- i) struct array*
- ii) ele (int)

OUTPUT: None

1. for (int i=a->size;i>=0;i--)
{
 int temp = a->arr[i];
 a->arr[(i*2)+1]=temp;
}
2. a->size = a->size*2;
3. for (int i=0;i<a->size;i=i+2)
{
 a->arr[i]=ele;
}

4. FUNCTION NAME: search

INPUT:

- i) struct array*
- ii) key (int)

OUTPUT: int

1. int pos = 1;
2. for (int i = 0; i < a->size; i++) {
 if (a->arr[i] == ele) {
 break;
 }
 pos = pos + 1;
}
3. if (pos == a->size + 1)
 return -1;
4. else
 return pos;

5. FUNCTION NAME: printarray

INPUT:

- i) struct array*

OUTPUT: None

1. for (int i = 0; i < a->size; i++) {
 DSIPLAY a->arr[i];
}

6.FUNCTION NAME:peak

INPUT:

i)struct arr*

OUTPUT: None

```
1. int arr1[a->size];
2. int k=0;
3. for (int i=0;i<a->size;i++)
{
    if (i==0)
    {
        if (a->arr[i+1]<a->arr[i])
        {
            arr1[k]=a->arr[i];
            k++;
        }
    }
    else if (i>0 && i<a->size)
    {
        if ((a->arr[i-1]<a->arr[i])&& (a->arr[i+1]<a->arr[i]))
        {
            arr1[k]=a->arr[i];
            k++;
        }
    }
    else
    {
        if (a->arr[i-1]<a->arr[i])
        {
            arr1[k]=a->arr[i-1];
            k++;
        }
    }
}
```

CODE:

arradt.h

```
#include <stdio.h>
struct array {
    int size;
    int arr[100];
};
void create(struct array *a, int s, int arr[]) {
    a->size = s;
    for (int i = 0; i < a->size; i++) {
        a->arr[i] = arr[i];
    }
}
```

```
}  
}
```

```
void peak(struct array *a)  
{  
    int arr1[a->size];  
    int k=0;  
    for (int i=0;i<a->size;i++)  
    {  
        if (i==0)  
        {  
            if (a->arr[i+1]<a->arr[i])  
            {  
                arr1[k]=a->arr[i];  
                k++;  
            }  
        }  
        else if (i>0 && i<a->size)  
        {  
            if ((a->arr[i-1]<a->arr[i])&& (a->arr[i+1]<a->arr[i]))  
            {  
                arr1[k]=a->arr[i];  
                k++;  
            }  
        }  
        else  
        {  
            if (a->arr[i-1]<a->arr[i])  
            {  
                arr1[k]=a->arr[i-1];  
                k++;  
            }  
        }  
    }  
    create(a,k,arr1);  
}
```

```
void display(struct array *a) {  
    for (int i = 0; i < a->size; i++) {  
        printf("%d ", a->arr[i]);  
    }  
    printf("\n");  
}
```

```
void insertfront(struct array *a, int front) {  
  
    for (int i = a->size - 1; i >= 0; i--) {  
        int temp = a->arr[i];  
        a->arr[i + 1] = temp;  
    }  
}
```

```
a->arr[0] = front;
a->size = a->size + 1;
}
```

```
void insertmiddle(struct array *a, int middle, int ind) {
    for (int i = a->size - 1; i >= ind; i--) {
        int temp = a->arr[i];
        a->arr[i + 1] = temp;
    }
    a->arr[ind] = middle;
    a->size = a->size + 1;
}
```

```
void insertend(struct array *a, int end) {
    a->arr[a->size] = end;
    a->size = a->size + 1;
}
```

```
void delete (struct array *a, int del) {
    int pos;
    for (int i = 0; i < a->size; i++) {
        if (a->arr[i] == del) {
            break;
        } else
            pos = pos + 1;
    }
    for (int i = pos; i < a->size; i++) {
        a->arr[i] = a->arr[i + 1];
    }
    a->size = a->size - 1;
}
```

```
int search(struct array *a, int ele) {
    int pos = 1;
    for (int i = 0; i < a->size; i++) {
        if (a->arr[i] == ele) {
            break;
        } else
            pos = pos + 1;
    }
    if (pos == a->size + 1)
        return -1;
    else
        return pos;
}
```

```
void insertevery(struct array *a,int ele)
```

```
{
for (int i=a->size;i>=0;i--)
{
    int temp = a->arr[i];
    a->arr[(i*2)+1]=temp;
}
a->size = a->size*2;
for (int i=0;i<a->size;i=i+2)
{
    a->arr[i]=ele;
}
}
```

ques1.c

```
#include "arradt.h"
#include <stdio.h>
#include <stdlib.h>

void main() {
    struct array *a;
    a = (struct array *)malloc(sizeof(struct array));
    printf("enter the size :");
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) {
        printf("enter the element : ");
        int ele;
        scanf("%d", &ele);
        arr[i] = ele;
    }
    create(a, n, arr);
    printf("\n---DISPLAYING---\n");
    display(a);
    printf("\n---INSERT BEFORE EVERY ELEMENT --\n");
    printf("enter the element :");
    int elt;
    scanf("%d", &elt);
    insertevery(a, elt);
    display(a);
    printf("Enter the element to be inserted at the front \n");
    int front;
    scanf("%d", &front);
    insertfront(a, front);
    printf("array after insertion at first place\n");
    display(a);
}
```

```
printf("\n");
printf("\n");
printf("enter the element to be deleted :");
int del;
scanf("%d", &del);
delete(a, del);
printf("\n");
printf("array after deletion :\n");
display(a);
printf("\n");
printf("enter the element to be searched : ");
int ele;
scanf("%d", &ele);
int res = search(a, ele);
if (res == -1)
    printf("element not found!");
else
    printf("element found at position %d", res);
printf("\n---PEAK--\n");
peak(a);
display(a);
}
```

OUTPUT:

```
enter the size :5
enter the element : 12
enter the element : 56
enter the element : 43
enter the element : 21
enter the element : 67

---DISPLAYING---
12 56 43 21 67

---INSERT BEFORE EVERY ELEMENT --
enter the element :90
90 12 90 56 90 43 90 21 90 67

enter the element to be deleted :12

array after deletion :
90 90 56 90 43 90 21 90 67

enter the element to be searched : 21
element found at position 7
---PEAK--
90 90 90
```


NAME: S.KARTHIKEYAN
CSE - B

3122 22 5001 056